Assignment 1 Data Structure

Problem 1

Q1:

```
int hello_recursive(int n) {
if (n <= 1) return 1; return
hello_recursive(n-1) +
hello_recursive(n-2); }
```

Complexity :

2^n

justification:

Each function call generates two recursive calls, forming a binary recursion tree.
Since the depth is about n and the number of nodes doubles at each level, the time complexity is O(2^n).

Problem 1

Q2:

```
void test(int n) { int i = n;
while (i > 1) { int j = 1;
while (j < i) { j *= 2; } i /= 2;
}}
```

Complexity :

O((log n)²).

justification:

The inner loop runs in $O(\log\ i)$because j is doubled each time until it reaches i.
The outer loop runs in $O(\log\ n)$since i is halved each iteration, giving a total time complexity of O((log n)²).

Problem 2

Complexity :

The best case is O(1) , it occurs when the target is found at the middle element.

The average case is O(log n) as the search space is halved each iteration.

The worst case is O(n) due to duplicates, where the algorithm may degrade to linear search.

Space complexity: O(1)